Qual é o objetivo principal da Integração Contínua (CI)?

A.

Automatizar a implementação de software diretamente no ambiente de produção.

B.

Mesclar com frequência alterações de código num repositório partilhado, validando-as com testes automáticos.

C.

Gerir a infraestrutura do servidor através de código (IaC).

D.

Substituir a necessidade de programadores realizarem testes unitários manuais.

Qual é a diferença fundamental entre Entrega Contínua (Continuous Delivery) e Implementação Contínua (Continuous Deployment)?

A.

Não existe diferença; os termos são sinónimos no DevOps.

B.

A Entrega Contínua requer uma intervenção manual para o lançamento em produção, enquanto a Implementação Contínua é automática.

C.

A Entrega Contínua não utiliza pipelines de automação.

D.

A Implementação Contínua foca-se apenas em ambientes de teste.

Comparativo Rápido

Característica	Entrega Contínua	Implementação Contínua
Testes Automatizados	Sim	Sim
Deploy para Produção	**Manual** (decisão humana)	**Automático**
Principal Benefício	Controle e confiança no release	Velocidade e feedback imediato
Pré-requisito	Pipeline robusto	Pipeline robusto + Alta cobertura de testes

---

Num pipeline de CI/CD, o que representa o termo 'Pipeline'?

A.

Uma linguagem de programação utilizada para automação.

B.

Uma ferramenta específica de chat para comunicação entre equipas.

C.

Um conjunto de passos automatizados que o código percorre desde a submissão até à produção.

D.

O hardware físico onde o código é guardado.

Qual destes é um benefício direto da adoção de práticas de CI/CD?

A.

Garantia de que o código nunca terá bugs.

B.

Eliminação total da necessidade de comunicação entre programadores e operações.

C.

Redução do risco em cada lançamento através de atualizações menores e frequentes.

D.

Aumento do tempo necessário para realizar uma build.

O que acontece normalmente na fase de 'Build' de um pipeline de CI?

A.

Os servidores são desligados para manutenção.

B.

O gestor de projeto aprova as novas funcionalidades.

C.

O código é compilado e as dependências são resolvidas para gerar um artefato executável.

D.

O software é vendido aos clientes finais.

O que caracteriza um 'Staging Environment' num pipeline de CD?

A.

Um ambiente isolado onde apenas são realizados testes unitários.

B.

Uma réplica exata do ambiente de produção usada para testes finais antes do lançamento.

C.

É o computador pessoal do programador onde o código é escrito.

D.

O servidor onde o código-fonte é armazenado permanentemente.

Qual é a importância dos 'Testes de Regressão' num pipeline automatizado?

A.

Testar a velocidade de escrita dos programadores.

B.

Validar apenas o design visual da aplicação.

C.

Apagar o histórico de commits do repositório.

D.

Garantir que novas alterações não quebraram funcionalidades que já funcionavam anteriormente.

Qual comando é utilizado para listar os arquivos e subdiretórios em diretório, incluindo os arquivos ocultos?

A.

show hidden

B.

list --all

C.

ls -l

D.

ls -a

Para mudar o proprietário de um arquivo no Linux, qual comando deve ser utilizado?

A.

chown

B.

chmod

C.

own

D.

Chgrp

Qual comando permite visualizar o conteúdo de um arquivo de texto diretamente no terminal sem abrir um editor?

A.

touch

B.

cat

C.

view

D.

Mkdir

Como se apaga um diretório que não está vazio através da linha de comandos?

A.

delete --all

B.

remove -dir

C.

rm -r

D.

Rmdir

Qual comando é utilizado para verificar o caminho completo do diretório onde se encontra atualmente?

A.

pwd

B.

cd

C.

whereami

D.

path

Qual é o comando utilizado para pesquisar padrões de texto dentro de arquivos?

A.

locate

B.

search

C.

find

D.

grep

Para dar permissão total (leitura, escrita e execução) a todos os usuários num arquivo, qual comando chmod deve ser usado?

A.

chmod +x

B.

chmod 666

C.

chmod 777

D.

chmod full

Como se pode criar um arquivo vazio chamado 'notas.txt' de forma rápida?

A.

new notas.txt

B.

cp notas.txt

C.

touch notas.txt

D.

mkfile notas.txt

Qual comando é usado para mover ou renomear arquivos e diretórios?

A.

mv

B.

rn

C.

move

D.

cp

Qual é a principal diferença entre uma Imagem Docker e um Container Docker?

A.

Uma imagem é uma instância em execução, enquanto um Container é o modelo estático armazenado no disco.

B.

Imagens são usadas apenas em ambiente de desenvolvimento e Containeres apenas em produção.

C.

Não há diferença técnica, são apenas nomes diferentes para o mesmo conceito de virtualização.

D.

Uma imagem é um modelo de leitura apenas (read-only), enquanto um Container é uma instância executável baseada nessa imagem.

Qual é o objetivo principal do arquivo 'Dockerfile'?

A.

Listar todos os Containers que estão atualmente em execução no host.

B.

Gerir as permissões de utilizador dentro do Docker Hub.

C.

Monitorizar o desempenho dos Containers em tempo real.

D.

Contém as instruções necessárias para criar automaticamente uma imagem Docker.

No Docker, para que servem os 'Volumes'?

A.

Para limitar a quantidade de memória RAM que um Container pode utilizar.

B.

Para persistir dados gerados por um Container além do seu ciclo de vida.

C.

Para isolar o tráfego de rede entre diferentes Containers.

D.

Para aumentar a velocidade de processamento da CPU do Container.

Qual comando é utilizado para baixar uma imagem do Docker Hub para a sua máquina local sem necessariamente instanciar um container com base nela?

A.

docker build (constrói um imagem a partir do Dockerfile)

B.

docker run (instancia o container, se necessário, baixa a imagem)

C.

docker pull

D.

docker push (envia imagem para o Docker Hub)

Qual é a principal diferença entre um Container Docker e uma Máquina Virtual (VM)?

A.

As VMs inicializam mais rapidamente do que os Containers Docker.

B.

Os Containers partilham o kernel do sistema operacional anfitrião (host), enquanto as VMs incluem um SO completo.

C.

Os Containers Docker requerem um Hypervisor como o VMware ou VirtualBox.

D.

Não existe diferença de performance entre ambos.

Qual é o comando utilizado para criar uma imagem a partir de um arquivo 'Dockerfile'?

A.

docker create

B.

docker run

C.

docker build

D.

docker push

No Docker, o que acontece aos dados guardados dentro de um Container quando este é removido, caso não tenham sido utilizados volumes?

A.

Os dados são perdidos permanentemente.

B.

Os dados são movidos automaticamente para o Docker Hub.

C.

O Docker impede a remoção do Container se houver arquivos novos.

D.

Os dados persistem no disco do anfitrião (host) para sempre.

Qual instrução do Dockerfile define o comando padrão que será executado quando o Container for iniciado?

A.

FROM

B.

RUN

C.

CMD

D.

EXPOSE

Qual é a principal diferença entre o comando 'docker run' e 'docker start'?

A.

O 'docker start' é utilizado apenas para containers em modo 'detached'.

B.

O 'docker run' cria e inicia um novo container a partir de uma imagem, enquanto o 'docker start' inicia um container que já existe e está parado.

C.

O 'docker run' apenas descarrega a imagem, enquanto o 'docker start' executa o processo principal.

D.

Não há diferença, são sinónimos utilizados em versões diferentes do Docker.

No contexto de um Dockerfile, para que serve a instrução 'EXPOSE'?

A.

Para indicar em que portas o container escuta em tempo de execução, servindo como documentação.

B.

Para definir a variável de ambiente da porta de rede da aplicação.

C.

Para abrir automaticamente a porta no host e mapeá-la para o mundo exterior.

D.

Para permitir que o container aceda à rede internet do host.

O que acontece se for executado o comando 'docker run -d nginx'?

A.

O container é executado em segundo plano (background).

B.

O Docker apaga o container imediatamente após a execução (deletable).

Obs.: --rm

C.

O container é executado com privilégios de administrador (debug mode).

D.

O container é executado em modo interativo com um terminal aberto.

Obs.: -it

Obs.: nginx?

O **NGINX** (pronuncia-se *"Engine-X"*) é um software de código aberto multifuncional, amplamente utilizado como **servidor web de alto desempenho**, **proxy reverso**, **balanceador de carga** (*load balancer*) e **gateway de API**.

Lançado originalmente por Igor Sysoev em 2004 para resolver o problema *C10K* (lidar com 10.000 conexões concorrentes simultâneas em um único servidor), o NGINX tornou-se um dos pilares da infraestrutura de software moderna na Web, impulsionando a maior parte dos sites e serviços de alto tráfego no mundo.

Arquitetura e Funcionamento

Diferente dos servidores web tradicionais que utilizam um modelo baseado em *threads* ou processos para cada requisição (como o Apache em suas configurações padrão legadas), o NGINX adota uma **arquitetura assíncrona e orientada a eventos** (*event-driven*).

- **Consumo Mínimo de Recursos:** Um único processo *worker* do NGINX consegue processar milhares de conexões HTTP simultâneas com uso extremamente baixo de memória RAM e CPU.
- **Processamento Não-Bloqueante:** Ele lida com I/O (entrada e saída) sem travar a execução, o que o torna ideal para servir arquivos estáticos e intermediar tráfego.

Principais Casos de Uso

Função	Descrição
Proxy Reverso	Fica à frente dos servidores de aplicação (Node.js, Java/Spring Boot, Python/FastAPI, PHP-FPM) para receber as requisições dos clientes e repassá-las internamente, ocultando a topologia da rede e provendo segurança.
Balanceador de Carga	Distribui o tráfego recebido entre múltiplos contêineres ou servidores de *backend* utilizando algoritmos como *Round Robin*, *Least Connections* ou *IP Hash*, garantindo alta disponibilidade e tolerância a falhas.
Servidor de Arquivos Estáticos	Entrega arquivos HTML, CSS, JavaScript, imagens e vídeos diretamente do sistema de arquivos com altíssima performance e sem sobrecarregar o código da aplicação.
Terminação TLS/SSL	Gerencia a criptografia HTTPS no nível da borda, aliviando os servidores de aplicação do custo computacional de cifrar e decifrar o tráfego.

Função	Descrição
API Gateway & Ingress Controller	Em arquiteturas de microserviços e ecossistemas **Kubernetes** (via NGINX Ingress Controller), atua como porta de entrada para roteamento de rotas, controle de taxa (*rate limiting*), autenticação e logs.

Qual é a vantagem de utilizar 'Multi-stage builds' num Dockerfile?

A.

Reduz o tamanho da imagem final ao separar o ambiente de compilação do ambiente de execução, copiando apenas o necessário.

B.

Torna obrigatório o uso de Docker Compose para construir a aplicação.

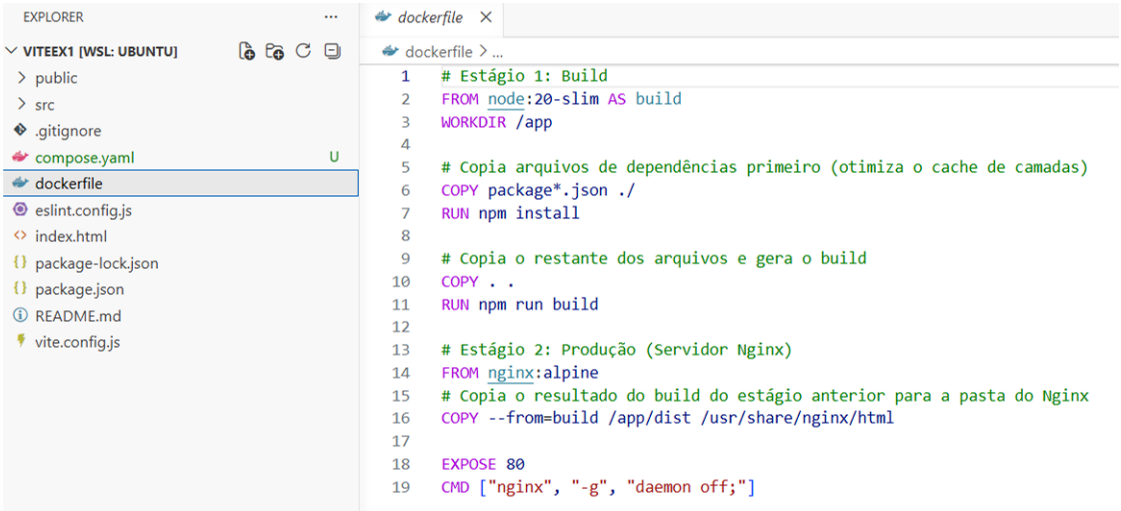
C.

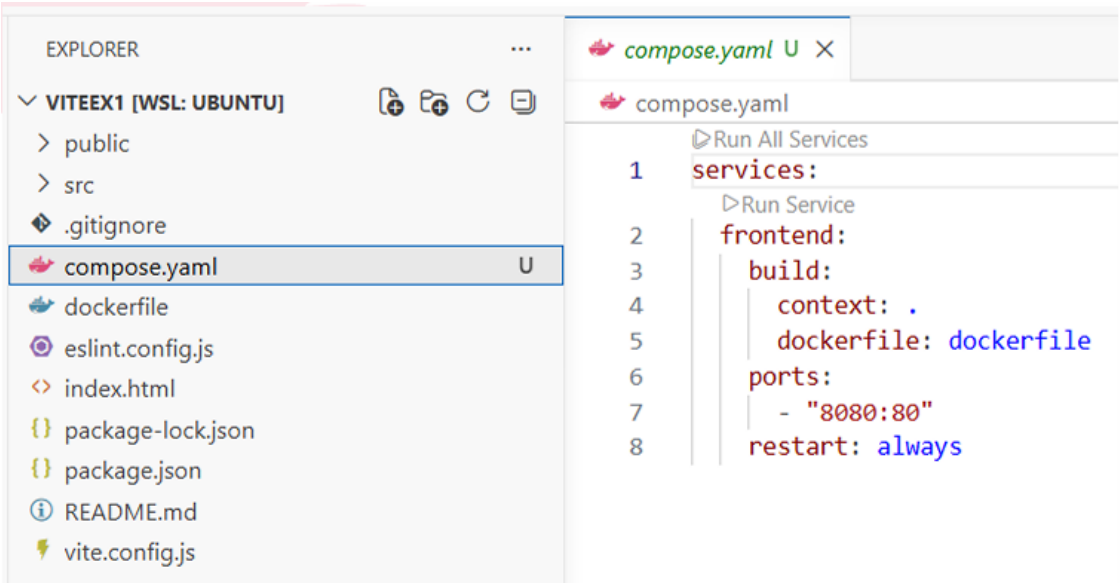
Aumenta a velocidade de download das imagens base originais.

D.

Permite executar múltiplos Containers ao mesmo tempo a partir do mesmo Dockerfile.

Exemplo Multi-stage





Qual comando é utilizado para criar um novo repositório Git local num diretório existente?

A.

git init

B.

git start

C.

git clone

D.

git create

Após alterar um arquivo, qual comando é usado para adicioná-lo à 'Staging Area' antes do commit?

A.

git push

B.

git add

C.

git commit

D.

git stage

Como se pode verificar o estado atual do repositório, vendo quais arquivos foram modificados ou estão na staging area?

A.

git log

B.

git status

C.

git show

D.

git check

Qual comando cria um novo 'branch' (ramo) e muda para ele imediatamente?

A.

git merge novo-ramo

B.

git branch novo-ramo

C.

git checkout -b novo-ramo

D.

git switch novo-ramo

Para baixar as alterações de um repositório remoto e mesclá-las automaticamente no seu ramo (Branch) local atual, qual comando usa?

A.

git fetch

B.

git pull

C.

git push

D.

git remote -v

Como se visualiza o histórico completo de commits de um projeto?

A.

git history

B.

git status

C.

git log

D.

git list

Qual comando é usado para enviar as suas alterações locais confirmadas para o repositório remoto?

A.

git commit

B.

git remote sync

C.

git upload

D.

git push

Qual comando mescla o conteúdo de um ramo específico (ex: 'feature') no ramo atual (ex: 'main')?

A.

git pull feature

B.

git join feature

C.

git merge feature

D.

git combine feature

Qual é a função do comando 'git remote add origin '?

A.

Cria um novo repositório no GitHub automaticamente.

B.

Faz o download imediato de todo o código do servidor.

C.

Apaga todas as versões anteriores do código no servidor.

D.

Vincula o seu repositório local a um repositório remoto específico, dando-lhe o nome de 'origin'.

Se apagou acidentalmente um arquivo e ainda não fez commit, qual comando (nas versões mais recentes do Git) é recomendado para restaurar esse arquivo?

A.

git back

B.

git restore <nome_do_arquivo>

C.

git clean -f

D.

git remove –undo

Sobre o GITHUB ACTION, responda as questões a seguir.

Questão 1

Em qual diretório do repositório e com qual extensão os arquivos de *workflow* do GitHub Actions devem ser salvos?

- **A)** .github/pipelines/ com extensão .json
- **B)** .github/workflows/ com extensão .yml

- **C)** config/actions/ com extensão .xml
- **D)** .workflows/github/ com extensão .sh

Questão 2

Por padrão, de que maneira múltiplos *jobs* definidos dentro de um mesmo arquivo de *workflow* são executados se nenhuma dependência for especificada?

- **A)** Em sequência, na ordem em que aparecem no arquivo.
- **B)** De forma aleatória, dependendo da disponibilidade dos servidores.
- **C)** Em paralelo, executando simultaneamente.
- **D)** Apenas o primeiro *job* executa, cancelando os demais.

Questão 3

Qual palavra-chave do YAML é utilizada na definição de um *job* para indicar que ele depende da conclusão bem-sucedida de outro *job* antes de iniciar?

- **A)** depends_on
- **B)** after
- **C)** requires
- **D)** needs

Questão 4

Como o GitHub Actions gerencia os dados armazenados na seção de **Secrets** do repositório para garantir a segurança?

- **A)** Salva em texto claro e permite visualização a qualquer momento na interface do GitHub.
- **B)** Criptografa os valores e exibe *** nos logs caso algum comando tente exibi-los.
- **C)** Permite acesso apenas para usuários com perfil de administrador do sistema operacional.
- **D)** Armazena os valores em arquivos públicos no diretório .github/workflows.

Questão 5

Qual evento (on:) permite adicionar um botão chamado **"Run workflow"** na interface do GitHub para disparar a automação manualmente?

- **A)** manual_trigger

- **B)** repository_dispatch
- **C)** workflow_dispatch
- **D)** push

Questão 6

Qual função de status em condicionais (if:) garante que uma etapa de limpeza (*cleanup*) seja executada **sempre**, independentemente de sucesso, falha ou cancelamento das etapas anteriores?

- **A)** always()
- **B)** success()
- **C)** failure()
- **D)** cancelled()

Questão 7

O que a expressão POSIX cron '0 0 * * *' configura ao ser utilizada em um evento agendado (schedule)?

- **A)** Executa o *workflow* a cada 15 minutos.
- **B)** Executa o *workflow* toda segunda-feira às 08:00 (UTC).
- **C)** Executa o *workflow* todo dia à meia-noite (UTC).
- **D)** Executa o *workflow* no dia 1º de cada mês ao meio-dia.

Obs.:

A sintaxe de uma expressão POSIX cron é dividida em 5 campos separados por espaço:

Minuto Hora Dia do Mês Mês Dia da Semana

Analisando a expressão '0 0 * * *':

- **0** (Minuto): No minuto **0**.
- **0** (Hora): Na hora **0** (00:00, ou seja, meia-noite).
- ***** (Dia do Mês): Em **qualquer** dia do mês.
- ***** (Mês): Em **qualquer** mês do ano.
- ***** (Dia da Semana): Em **qualquer** dia da semana.

Questão 8

Qual é a principal diferença de visibilidade e segurança entre **Variáveis de repositório** e **Secrets**?

- **A)** Variáveis usam criptografia, enquanto Secrets são exibidos em texto claro nos logs.
- **B)** Variáveis armazenam dados não sensíveis visíveis em texto claro, enquanto Secrets são criptografados e mascarados.
- **C)** Secrets funcionam apenas no escopo global, e Variáveis funcionam apenas dentro de um *step*.
- **D)** Não há diferença prática de segurança entre ambos.

Questão 9

Para configurar uma etapa de notificação ou envio de logs que deve ser executada **apenas se pelo menos uma etapa anterior do mesmo job falhar**, qual função de status deve ser usada?

- **A)** success()
- **B)** failure()
- **C)** always()
- **D)** cancelled()

Questão 10

Em relação à sintaxe dos arquivos YAML no GitHub Actions, qual é a regra quanto à identação?

- **A)** Deve-se utilizar obrigatoriamente a tecla TAB para alinhar os blocos.
- **B)** Não é necessário manter identação no formato YAML.
- **C)** Deve-se utilizar identação com espaços em branco (ex.: 2 espaços) e não utilizar a tecla TAB.
- **D)** A identação só é exigida nas linhas iniciadas com - name:.